

Exercício 1: Gerenciamento de Camisas

Objetivo: Praticar a criação de classes em Python, uso do construtor `__init__`, atributos de instância e métodos com validação condicional.

Contexto:

Você foi encarregado de desenvolver o módulo de controle de camisas para o sistema de uma loja de roupas. O sistema deve permitir o cadastro de novas peças, a visualização dos dados e o registro de vendas com baixa no estoque.

Requisitos:

1. Crie a classe Camisa.
2. No método construtor `__init__`, defina e inicialize os seguintes atributos:
 - cor (texto)
 - tamanho (texto: "P", "M", "G", "GG")
 - preco (número decimal)
 - tipo_gola (texto: ex: "Gola Polo", "Gola V", "Gola Redonda")
 - quantidade_estoque (número inteiro)
3. Implemente os seguintes métodos na classe:
 - `exibir_informacoes()`: Mostra na tela todos os dados da camisa de forma organizada.
 - `vender(quantidade)`: Recebe a quantidade de peças a serem vendidas. Se houver estoque suficiente, realiza a subtração e exibe o estoque restante; caso contrário, exibe uma mensagem avisando que o estoque é insuficiente.
4. **Área de Teste:**
 - Crie um objeto da classe Camisa (ex: cor "Azul", tamanho "M", preço 79.90, gola "Polo", estoque 10).
 - Chame o método `exibir_informacoes()`.
 - Realize uma venda válida (ex: 3 unidades).
 - Tente realizar uma venda com quantidade maior do que o estoque disponível para testar a validação.

Exercício 2: Gerenciamento de Calças

Objetivo: Praticar a definição de classes, manipulação de valores numéricos em atributos e aplicação de cálculos por meio de métodos.

Contexto:

A loja de roupas precisa controlar o estoque de calças e aplicar remarcações de preço promocionais em peças paradas no estoque.

Requisitos:

1. Crie a classe Calca.
2. No método construtor `__init__`, defina e inicialize os seguintes atributos:
 - cor (texto)
 - tamanho (número inteiro: ex: 38, 40, 42)
 - preco (número decimal)
 - tipo_tecido (texto: ex: "Jeans", "Sarja", "Moletom")
 - quantidade_estoque (número inteiro)
3. Implemente os seguintes métodos na classe:
 - `exibir_informacoes()`: Exibe os detalhes da calça formatados na tela.
 - `aplicar_desconto(porcentagem)`: Recebe um valor de porcentagem (ex: 10 para 10%), calcula o desconto, atualiza o atributo `preco` da peça com o novo valor reduzido e exibe uma mensagem com o novo preço.
4. **Área de Teste:**
 - Crie um objeto da classe Calca (ex: cor "Preta", tamanho 42, preço 120.00, tecido "Jeans", estoque 8).
 - Chame o método `exibir_informacoes()`.
 - Aplique um desconto de 15% utilizando `aplicar_desconto(15)`.
 - Chame `exibir_informacoes()` novamente para confirmar a alteração permanente do preço.

Exercício 3: Gerenciamento de Blusões

Objetivo: Praticar o uso de atributos booleanos (Verdadeiro/Falso), instanciação de objetos e métodos para atualização e reposição de estoque.

Contexto:

Com a chegada do inverno, a loja começou a receber lotes de blusões. O sistema precisa registrar as características da peça (incluindo se possui capuz) e gerenciar as reposições periódicas de estoque.

Requisitos:

1. Crie a classe Blusao.
2. No método construtor `__init__`, defina e inicialize os seguintes atributos:
 - cor (texto)
 - tamanho (texto: "P", "M", "G", "GG")
 - preco (número decimal)
 - possui_capuz (booleano: True ou False)
 - quantidade_estoque (número inteiro)
3. Implemente os seguintes métodos na classe:
 - `exibir_informacoes()`: Mostra os detalhes do blusão, indicando textualmente se a peça possui capuz ou não, junto com preço e estoque.
 - `repor_estoque(quantidade)`: Recebe uma quantidade de peças que chegaram do fornecedor, soma ao estoque atual e mostra o total atualizado.
4. **Área de Teste:**
 - Crie um objeto da classe Blusao (ex: cor "Cinza", tamanho "G", preço 180.00, possui_capuz True, estoque 4).
 - Chame o método `exibir_informacoes()`.
 - Realize uma reposição de 6 peças utilizando o método `repor_estoque(6)`.
 - Chame `exibir_informacoes()` novamente para verificar o estoque final.